

Proyecto Final

Plataforma Colaborativa de Información Universitaria “CampusRadar”

Profesor: Jorge Zambrano Ibujés

Auxiliar: Christian Díaz Guerra

Ayudante: Pablo Vergara Llantén

1. Contexto y motivación

La vida universitaria es altamente dinámica. Todos los días ocurren eventos efímeros y situaciones relevantes para la comunidad estudiantil: sobra comida en una actividad, se libera una sala de estudio, un microondas deja de funcionar, hay congestión en un sector del campus, o se produce un corte de agua o electricidad.

Actualmente, gran parte de esta información circula de manera informal mediante chats, grupos de mensajería, redes sociales o foros poco estructurados, dificultando encontrar información relevante de forma rápida y confiable.

El objetivo de este proyecto es desarrollar una plataforma web colaborativa denominada CampusRadar (similar a **SOSAFE** o **Waze**), orientada a reportar, validar y visualizar información del campus en tiempo real.

El sistema debe modelar entidades con comportamiento dinámico, ciclos de vida, validación comunitaria, reputación de usuarios y mecanismos de interacción social.

2. Objetivos

Los objetivos de este proyecto son:

- Diseñar un sistema complejo utilizando Programación Orientada a Objetos.
- Aplicar correctamente: herencia, polimorfismo, abstracción, encapsulamiento, composición.
- Modelar un dominio real con múltiples reglas de negocio.
- Diseñar una arquitectura mantenible y extensible.
- Implementar tests automatizados.
- Separar responsabilidades entre dominio, lógica de negocio, persistencia y presentación.
- Desplegar una aplicación web funcional.
- Utilizar herramientas de IA de manera efectiva para apoyar el desarrollo del sistema.

3. Descripción del proyecto

Se deberá implementar una plataforma web colaborativa para administrar y visualizar reportes comunitarios relacionados con la vida universitaria.

El sistema deberá permitir:

- Gestionar usuarios y permisos.
- Crear y administrar reportes.
- Confirmar y desmentir información.
- Comentar publicaciones.
- Gestionar reputación y validación comunitaria.
- Buscar y filtrar información.
- Clasificar publicaciones mediante tags.
- Administrar estados y ciclos de vida de los reportes.
- Visualizar reportes geolocalizados.
- Implementar mecanismos de moderación.
- Gestionar notificaciones o suscripciones.

El foco principal del proyecto está en el diseño del backend, la arquitectura del sistema, y el modelado orientado a objetos. No se espera una solución única. Parte importante de la evaluación considera las decisiones de arquitectura y modelado tomadas por cada grupo. **La interfaz web no necesita ser visualmente compleja, pero sí debe ser funcional y usable.**

3.1. Reportes y ciclos de vida

El núcleo principal del sistema corresponde a los reportes realizados por la comunidad. Todo reporte debe modelarse a partir de una entidad base abstracta. El sistema debe soportar al menos distintos tipos de publicaciones, por ejemplo:

- Información de infraestructura.
- Actividades extraprogramáticas.
- Alertas de emergencia.
- Eventos universitarios.
- Información logística.

Cada reporte debe poseer autor, timestamp, ubicación, descripción, estado, métricas de validación, tags asociados, e interacciones de usuarios.

Uno de los focos principales del proyecto es el modelado del ciclo de vida de los reportes. Los reportes deben evolucionar dinámicamente en el tiempo según antigüedad, confirmaciones, desmentidos, reputación de los usuarios involucrados y otros criterios definidos por el grupo.

El sistema debe contemplar estados internos para los reportes. Por ejemplo: nuevo, verificado, controvertido, crítico, expirado, archivado.

El grupo deberá decidir cómo evolucionan los estados, cómo afecta el tiempo al sistema, qué reglas de validación existen, y cómo se determina la relevancia de un reporte. **No**

se permitirá utilizar mecanismos bloqueantes para simular el paso del tiempo (`time.sleep`, `loops` infinitos, etc.). Se espera modelar correctamente el problema utilizando mecanismos apropiados de diseño. Por ejemplo, se puede utilizar lazy evaluation para determinar si el reporte sigue activo.

3.2. Usuarios y permisos

El sistema debe contemplar distintos tipos de usuarios. Como mínimo, deben existir mecanismos de autenticación básica y permisos diferenciados. El grupo deberá modelar adecuadamente la jerarquía de usuarios. El sistema puede considerar actores tales como estudiantes, moderadores, administradores.

Los usuarios pueden crear reportes, comentar, confirmar información, desmentir publicaciones, denunciar contenido, seguir tags o zonas, y recibir notificaciones.

El sistema debe incluir algún mecanismo de reputación o credibilidad asociado a los usuarios.

3.3. Interacciones sociales

El sistema debe permitir interacción entre usuarios y publicaciones. Como mínimo, los reportes deben permitir: comentarios, confirmaciones, desmentidos, y denuncias.

Se espera modelar correctamente estas interacciones y sus relaciones con el sistema principal. El grupo puede decidir cómo representar las interacciones, cómo afectan el estado del reporte, y qué impacto tienen sobre reputación y visibilidad.

3.4. Tags, búsqueda y organización de información

Uno de los problemas principales que busca resolver el sistema es la organización de información. El sistema debe incluir mecanismos de clasificación y búsqueda. Las publicaciones deberán poder clasificarse mediante tags o categorías.

Se espera como mínimo: búsqueda por tags, filtrado de publicaciones, organización del feed, y visualización de información relevante. El grupo puede proponer: algoritmos de ranking, sistemas de relevancia, tendencias, suscripciones, o recomendaciones. Se evaluará la coherencia del diseño y la calidad de la solución implementada.

3.5. Geolocalización y campus

El sistema debe incluir geolocalización básica. No es necesario implementar navegación avanzada ni sistemas GIS complejos. Como mínimo, los reportes deben estar asociados a una ubicación o zona del campus.

Debido a que no se cuenta con un mapa detallada del campus, al momento de la creación del reporte se debe seleccionar una ubicación aproximada en el mapa, nombre del edificio (hall sur, biblioteca, etc.) y piso. El sistema debe permitir visualizar los reportes en un mapa y filtrar por ubicación.

El grupo puede utilizar herramientas externas tales como OpenStreetMap, Leaflet u otras bibliotecas equivalentes. No se evaluará complejidad gráfica avanzada.

3.6. Moderación y validación

El sistema debe contemplar mecanismos básicos de moderación. Como mínimo, debe existir: denuncia de contenido, validación de acciones, manejo de publicaciones inválidas, y restricciones para evitar estados inconsistentes.

El grupo puede definir: políticas de moderación, sanciones, restricciones de reputación, o flujos de revisión.

3.7. Testing

El proyecto debe incluir test automatizados. Como mínimo, tests unitarios, tests de lógica de negocio y validación de reglas importantes del sistema. Por ejemplo, comportamiento temporal, reputación, validaciones, etc.

3.8. Backend

El backend debe estar desarrollado en Python. Se recomienda el uso de FastAPI, SQLAlchemy y pytest, pero queda a criterio del grupo que stack utilizar, justificando su elección. La aplicación debe incluir persistencia de datos, autenticación básica y despliegue.

3.9. Persistencia de datos

La aplicación debe utilizar una base de datos. No se evaluará la complejidad avanzada del modelo relacional, pero sí que funcione adecuadamente: que sea consistente, con relaciones adecuadas y manejo correcto de datos.

3.10. Interfaz web

La aplicación debe incluir una interfaz web funcional. NO se evaluará diseño visual avanzado, solo que se sea funcional. Se espera como mínimo: navegación básica, visualización del mapa, formularios funcionales, visualización de información, interacción con publicaciones, búsqueda, filtros e interacción con el backend.

3.11. Despliegue

La aplicación debe poder ser desplegada fácilmente mediante Docker, sin instalación manual de dependencias. Debe poder mostrarse su funcionamiento, ya sea de forma local o pública. Puede utilizar servicios como: Render, Railway, Vercel u otros. El despliegue debe incluir aplicación funcional, acceso autenticado y persistencia operativa. NO se evaluarán componentes de seguridad informática. Se darán bonificaciones si se despliega en un servicio en la nube.

4. Uso de herramientas de IA

Está permitido el uso de herramientas de inteligencia artificial para apoyar el desarrollo del proyecto. Sin embargo, deben tener en cuenta que el objetivo principal no es únicamente

generar código, sino que también diseñar correctamente el sistema, tomar decisiones de arquitectura, modelar adecuadamente el dominio y justificar las soluciones implementadas.

La evaluación y revisión considerará la explicación de sus arquitecturas, justificación de decisiones de diseño, explicación de las relaciones entre clases, descripción de responsabilidades de cada componente y la comprensión completa del código entregado.

5. Entregables

- **Código fuente:** Repositorio en GitHub completo del proyecto. Debe incluir README con instrucciones de ejecución y despliegue, dockerfiles y estructura clara.
- **Prompts utilizados:** Cada uno de los prompts utilizados para la generación del código usando herramientas de IA, indicando a qué parte del código se utilizó.
- **Presentación final:** Deben realizar una presentación final la semana de exámenes, enfocándose en el problema abordado, las decisiones de diseño tomadas y la solución implementada. Se espera que cada grupo muestre el sistema funcionando en vivo, destacando los principales flujos de uso, la arquitectura general, las decisiones técnicas más relevantes y los desafíos enfrentados durante el desarrollo. La evaluación considerará tanto la calidad técnica del sistema como la capacidad del grupo para explicar y defender su solución de manera clara.